

Don't Trust the Facilitator: Chain-Anchored Settlement Verification for x402 Machine Payments with Free Hedera Mirror-Node Quorums

Dogukan Ali Gundogan
Independent Researcher, Munich

July 30, 2026

Abstract

The x402 protocol revives HTTP 402 as a payment rail for machine-to-machine commerce, but every deployed implementation delegates settlement to a centralized facilitator whose `SettleResponse` is accepted on faith. A compromised, buggy, or merely optimistic facilitator can fabricate settlements, short-pay, redirect funds, censor honest payers, or replay a single payment across many resource grants, and no shipped gateway checks the claim against chain state before releasing the resource. We present a resource-server-side auditor that treats the facilitator as untrusted: each claimed settlement is re-verified against a 2-of- N quorum ($N = 3$ in our evaluation) of independent Hedera mirror nodes, cryptographically bound to the HTTP request through a canonical memo digest, screened against an idempotent spend ledger, and certified with a signed JSON Web Signature (JWS) fraud receipt that any third party can replay offline (measured under the artifact's symmetric test key, which establishes replayability and tamper detection but not unforgeability). Across ten independently seeded runs of a 5,100-settlement simulation (3,000 honest, 2,100 adversarial spread over six lie families, i.e. distinct classes of facilitator misreport), the auditor blocks all 21,000 adversarial cases (detection rate 1.000 ± 0.000 across runs; pooled Wilson 95% CI [99.98%, 100%]) at a $0.067\% \pm 0.047\%$ false-positive rate (pooled 20/30,000, Wilson 95% CI [0.043%, 0.103%]) and a $0.057\% \pm 0.042\%$ false fraud-accusation rate (pooled 17/30,000, CI [0.035%, 0.091%]), versus 0% detection for the trust-the-facilitator baseline. Median added latency is 1.71 s (SD 6 ms across runs) with 2.01 mirror calls per settlement at zero marginal query cost, against 5.54 s and \$0.101 per thousand verifications for an Ethereum Virtual Machine (EVM) receipt-polling control.

1 Introduction

Agentic commerce needs a payment primitive that a program can execute without a human in the loop. The x402 protocol answers with a minimal design: a resource server responds 402 Payment Required with machine-readable

`PaymentRequirements`, the client attaches a signed authorization in an `X-PAYMENT` header, and a facilitator service verifies and settles the transfer on-chain, returning a `SettleResponse` that the gateway echoes as `X-PAYMENT-RESPONSE`. The design is elegant because it removes wallet and remote procedure call (RPC) plumbing from the resource server. It is also, as deployed, a pure trust delegation: the gateway releases a paid resource because an HTTP service said success: true.

That assertion carries no proof. Nothing in the response is a chain artifact the server independently checked; the transaction identifier it contains may name a transaction that failed, that paid a different recipient, that paid a smaller amount, that belongs to an unrelated request, or that does not exist at all. We call each such class of facilitator misreport a lie family. Recent adversarial analyses of x402 make the gap concrete: a Byzantine-facilitator harness demonstrated five distinct attacks on live deployments [6], and a survey of fifteen production facilitators found every one violating at least one of eight basic settlement-integrity rules [7]. The failure is not exotic cryptography; it is a missing read.

The read is missing for an economic reason. On EVM chains, confirming that a specific transfer of a specific amount to a specific recipient reached finality requires archive-node access or metered RPC credits, plus confirmation-depth waiting to survive reorganizations. Gateways skip verification precisely because it costs money and seconds on every request, and x402 payments are frequently sub-cent. Hedera changes this arithmetic. Its mirror nodes expose a free, keyless, public REST API over full transaction history, and its asynchronous Byzantine fault tolerant (aBFT) consensus delivers deterministic finality in roughly three seconds with no reorganization risk and therefore no confirmation-depth gating. Verification becomes an $O(1)$ free HTTP GET rather than a recurring RPC bill.

We exploit that asymmetry to build an auditor that never trusts the facilitator. Our contributions:

- A zero-trust settlement audit layer that gates resource release on independent chain evidence, verified through a bounded 2-of- N quorum ($N = 3$ in our evaluation) over heterogeneous mirror nodes

inside a hard budget of three HTTP calls, failing closed when no quorum is reached, so a single compromised or lagging mirror cannot mint a false grant.

- Canonical request-to-payment binding: a SHA-256 digest over the RFC 8785 JSON Canonicalization Scheme (JCS) serialization of PaymentRequirements—whose server-minted nonce, resource path and expiry make the digest unique to one request—carried as the unpadded base64url encoding of the 32-byte digest, which is exactly 43 characters and therefore fits inside Hedera’s 100-byte transfer memo, collapsing cross-request replay and payment-substitution attacks into a single comparison.
- An idempotent spend ledger keyed on transaction identifier and memo digest with insert-on-conflict evaluated before the response write, eliminating all 1,000 duplicate grants that the same sequential replay trace produces without it; concurrent interleaving of replays is not exercised by our evaluation.
- Portable, offline-replayable fraud receipts emitted as a signed JWS in an X-PAYMENT-RECEIPT header, achieving 100% replay agreement over an 800-receipt sample and 100% tamper rejection over 200 single-field mutations drawn from the 2,103 receipts the primary run issues, so a third party who trusts neither the server nor the facilitator can recompute the verdict from the receipt alone; both arms were measured under the artifact’s symmetric HS256 test key, so they establish replayability and tamper detection rather than non-repudiation, which requires the asymmetric production signing configuration we do not evaluate.
- A six-lie-family adversarial evaluation with ablations A_0 – A_5 tested by exact McNemar and replicated over ten independent master seeds, and a cost/latency comparison against an EVM receipt-polling control arm.

2 Related Work

The x402 protocol and its facilitators. The Coinbase x402 specification and reference facilitator define the 402/X-PAYMENT handshake and the SettleResponse contract, but leave settlement verification entirely to the facilitator [1, 2]. Independent implementations—notably the Rust x402-rs facilitator with multi-chain support and OpenTelemetry instrumentation [3], and the express and Next.js middleware packages [4]—improve operability and observability without changing the trust model: settlement remains self-asserted. Hedera-targeted facilitators [5] inherit the same property. Our work is complementary rather than competing: we leave the facilitator in place and add a verifier downstream of it.

Adversarial analysis of x402. Li, Wang and Wang enumerate five attacks against x402 deployments using a Byzantine-facilitator harness and sketch mitigations M2/M4 [6]; a USENIX Security 2026 audit of fifteen facilitators against eight settlement rules reports universal violation [7]. These establish that the threat is empirical, not hypothetical. Both stop at measurement and mitigation sketches; we contribute a shipped-shaped verifier, a quantified false-positive budget on a 3,000-settlement honest corpus replicated ten times, and receipts that make a verdict transferable.

Payment-channel and micropayment integrity. Lightning [8], Perun-style virtual channels [9], and probabilistic micropayments [10] all reduce per-payment chain cost by moving trust into cryptographic constructions rather than an intermediary, but require stateful channel management that x402’s stateless HTTP model deliberately avoids. Earlier HTTP-native schemes—Millicent [11], NetBill [12], and the original HTTP 402 discussions [13]—anticipated the gateway trust problem without a public ledger to resolve it. We keep x402’s statelessness and resolve trust by reading a ledger that is free to read.

Verifiable off-chain claims and light verification. Light-client and proof-of-inclusion approaches [14, 15] minimize verification cost on Nakamoto-consensus chains but still assume header-chain synchronization and confirmation-depth heuristics. Hedera’s hashgraph consensus [16] provides deterministic finality, and its mirror-node architecture [17] externalizes full history as free REST, which is the specific enabling property we depend on. For accusation portability we rely on JWS [18] over JCS-canonicalized JSON [19]. Transparency-log designs [20] similarly convert trust into third-party-auditable evidence; our fraud receipts play the analogous role at the granularity of a single HTTP payment. Statistical treatment follows Wilson intervals [21] and the exact McNemar test [22].

3 Problem Formulation

Let a paid request be $r = (\rho, \nu)$ where ρ is the PaymentRequirements object (scheme, network, payee account, amount, asset, resource path, and expiry validUntil) and ν is the per-request nonce the resource server mints; because ν , the resource path and the expiry are themselves fields of ρ , one canonical digest over ρ identifies this request and no other. The facilitator returns a claim $c = (\text{tid}, s, \hat{a}, \hat{p}, \hat{m}, \hat{t})$: a transaction identifier, an asserted success bit, and asserted amount, payee, memo and consensus timestamp. Chain truth is a partial map $\mathcal{L} : \text{tid} \mapsto (a, p, m, t, \sigma)$ with σ the on-chain status; a claim is sound iff $\mathcal{L}(\text{tid})$ exists and agrees with r .

We define the canonical binding

$$\mu(r) = \text{b64u}_{43}(\text{SHA-256}(\text{JCS}(\rho))), \quad (1)$$

where `b64u43` is the unpadding base64url encoding of the 32-byte SHA-256 digest—exactly 43 characters, and therefore comfortably inside Hedera’s 100-byte memo field with no truncation of the digest—so that a memo m satisfying $m = \mu(r)$ certifies that the transfer was authorized for this request and no other. The auditor evaluates five predicates over an observed record ℓ , at the instant t_{dec} at which the verdict binds:

$$\begin{aligned} P_1(\ell) : \sigma &= \text{SUCCESS}, \\ P_2(\ell) : a &\geq \rho.\text{amount} \wedge p = \rho.\text{payee}, \\ P_3(\ell) : m &= \mu(r), \\ P_4(\ell) : -\sigma_{\text{skew}} &\leq t_{\text{dec}} - t \leq \Delta_{\text{fresh}} \\ &\wedge t \leq \rho.\text{validUntil} + \sigma_{\text{skew}}, \\ P_5(\ell) : \text{tid} &\notin \mathcal{S} \wedge \mu(r) \notin \mathcal{S}, \end{aligned} \quad (2)$$

where $\mathcal{S}$ is the spend ledger. Here Δ_{fresh} is the freshness window: the maximum tolerated age of a record at verdict time. A record older than Δ_{fresh} is not a plausible settlement for a live request, and one whose consensus timestamp lies in the future beyond the tolerated clock skew σ_{skew} is not a plausible settlement at all; P_4 rejects both, and additionally rejects any record settled after the requirement expired. Their values in all experiments are $\Delta_{\text{fresh}} = 180\text{s}$ and $\sigma_{\text{skew}} = 5\text{s}$, against requirements minted with a 120s validity.

A single mirror may be stale, faulty, or Byzantine, so the auditor queries a set M of N independent mirrors and admits a grant only under quorum agreement. Each mirror $j \in M$ returns one of three observations for tid : a record ℓ_j ; pending, meaning no record yet and the node’s head consensus timestamp head_j has not passed the claim; or the sentinel $\perp$ (absent), meaning no record and $\text{head}_j > \hat{t} + \varepsilon$ for a head margin ε , i.e. the node has provably ingested past the point where the record would have appeared. This three-way split, not a timeout, is what turns an HTTP 404 into evidence; $\varepsilon = 1.4\text{s}$ is fixed by the calibration sweep of Section 5. Writing $\phi(\cdot)$ for the record fingerprint (identifier, status, memo, consensus timestamp, transfer set) and ϕ^* for the fingerprint of the largest agreeing group, the set of corroborating mirrors is

$$\mathcal{C}(c, r) = \left\{ j \in M : \phi(\ell_j) = \phi^* \wedge \bigwedge_{i=1}^5 P_i(\ell_j) \right\}, \quad (3)$$

so that the grant decision is $V(c, r) = 1$ if $|\mathcal{C}(c, r)| \geq k$ and $V(c, r) = 0$ otherwise, with $k = 2$ and $N = 3$ throughout our evaluation. An adverse $V = 0$ is split by evidence: the auditor accuses fraud when k mirrors agree on absent or when the agreed record fails a predicate, and merely holds the resource when no k mirrors agree either way within the three-call budget. Writing F for the false-positive rate on honest traffic (any honest settlement not granted, accusations and holds alike), R for detection on the adversarial corpus, L for added latency and C for per-verification cost, the design objective is

$$\max_{k, N, \varepsilon, D, W} R \quad \text{s.t.} \quad F \leq F^*, L_{p95} \leq L^*, C = 0, \quad (4)$$

over the probe schedule (D, W) of Section 4, with the ledger constraint that for any replayed tid the number of grants is exactly one and the budget constraint of at most three mirror calls per settlement. We instantiate the two bounds of (4), fixed in advance of the reported runs, as $F^* = 0.005$ (at most one honest settlement in two hundred withheld) and $L_{p95}^* = 3,000\text{ms}$; these are the values used for every feasibility and acceptance decision reported below. The baseline gateway is the degenerate policy $V_{A_0}(c, r) = s$, which trivially satisfies $L = 0$ and $C = 0$ and yields $R = 0$.

4 Approach

The auditor sits between the x402 middleware and the resource handler and is structured as five cooperating modules (Fig. 1); the end-to-end settlement verification path those modules implement, from the initial 402 response to the grant-or-receipt decision, is shown in Fig. 2. Its governing rule is that no byte of a paid resource leaves the process until $V(c, r) = 1$ has been recorded durably.

Canonicalization layer. Binding is only as strong as its serialization. We canonicalize `PaymentRequirements` with RFC 8785 JCS [19] before hashing, so key ordering, Unicode normalization, and number formatting cannot produce two digests for one logical requirement—the exact ambiguity an adversarial facilitator would exploit to make a memo appear to match. The resulting $\mu(r)$, the 43-character unpadding base64url encoding of the full 32-byte digest, is what the client places in the transfer memo and what predicate P_3 recomputes locally; the auditor never accepts the facilitator’s assertion $\hat{m}$ about what the memo contained.

Mirror quorum client. `MirrorAuditor` issues two concurrent GETs to two of the $N = 3$ independent mirror endpoints at $t_1 = \max(t_{\text{recv}}, \hat{t} + D)$, where t_{recv} is the arrival of the `SettleResponse`, $\hat{t}$ the claimed consensus timestamp, and $D = 2.2\text{s}$ the pre-probe delay chosen by calibration; mirrors are separately operated and do not advance in lockstep, so each is modeled with its own ingestion-lag draw and its own head timestamp. (Throughout, k denotes only the agreement threshold of Eq. (3), not the number of probes issued.) A record that is absent may mean fraud or may mean lag, so the client distinguishes the two by comparing the node’s head timestamp against the claim: no record while $\text{head}_j \leq \hat{t} + \varepsilon$ is pending, and no record once $\text{head}_j > \hat{t} + \varepsilon$ is absent and admissible as evidence. This distinction is what keeps the false-positive rate near zero without weakening detection. There is no unbounded retry loop: if two mirrors do not already agree and a quorum is still arithmetically reachable, the auditor spends

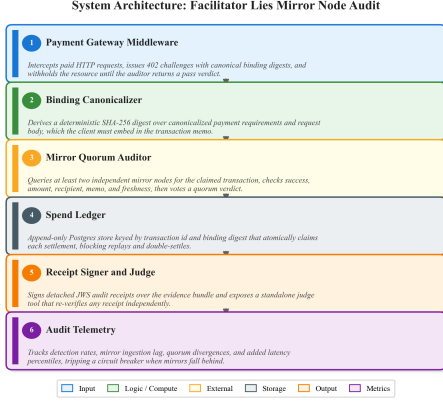


Figure 1: The five auditor modules and the flow of a claimed settlement between them: the canonicalization layer (JCS memo digest $\mu(r)$), the mirror quorum client (two concurrent GETs to two of the $N = 3$ mirrors at $\hat{t} + D$, with one tie-break probe at $\hat{t} + W$ and a hard cap of three calls), the predicate evaluator (P_1 – P_5 over the agreed record), the spend ledger (insert-on-conflict on transaction identifier and memo digest), and the receipt issuer (signed JWS in X-PAYMENT-RECEIPT). The diagram also sketches a deferred audit queue with re-checks at 15 s/60 s/600 s; that component is future work—it is neither implemented in the evaluated system nor exercised by any run reported here—and is not one of the five modules. Release of the paid resource is gated on $V(c, r) = 1$: no response bytes are written until the quorum decision is recorded durably.

its one remaining call on the third, independent node at $t_2 = \max(t_1 + \text{rtt}, \hat{t} + W)$ with $W = 3.4$ s, and otherwise stops immediately. The budget is a hard cap of three HTTP calls per settlement rather than a wall-clock deadline; if the third response still leaves fewer than k agreeing mirrors, the auditor fails closed with $V = 0$ and holds the resource without accusing anyone.

Predicate evaluator and spend ledger. The five predicates are evaluated over the agreed record with the freshness window fixed at $\Delta_{\text{fresh}} = 180$ s and the clock-skew tolerance at $\sigma_{\text{skew}} = 5$ s, and only a fully-passing record can produce a grant—an important detail, since a Byzantine mirror that returns a plausible-but-wrong record contributes nothing unless a second, independent node returns the identical fingerprint. The ledger performs INSERT ... ON CONFLICT DO NOTHING against uniqueness constraints on both the transaction identifier and the memo digest before the response write, so idempotency is decided by the database’s uniqueness guarantee rather than by application logic that a concurrent request could interleave past. That last property is a claim about the constraint, not a measured one: the evaluation of Section 5 runs at concurrency level one and therefore

never places two replays of the same identifier in flight together.

Receipts. Every adverse verdict emits a signed JWS receipt [18] in X-PAYMENT-RECEIPT plus an append-only log line, carrying the claim, the per-mirror observations, the failing predicate, and the classified failure mode. The x402-judge command-line interface (CLI) re-runs the same predicates over a receipt offline; because the receipt embeds the evidence, a third party who trusts neither party can reproduce the verdict, and—under the asymmetric production signing configuration of Section 5—attribute it to the issuing auditor. In the evaluated system every verdict binds synchronously: a claim that reaches no quorum is held and counted as a false positive, which is the conservative accounting. A deferred re-check queue for optimistically granted payments—re-examining unresolved claims at 15 s, 60 s and 600 s and closing anything still unresolved at the terminal re-check as fraudulent—is a natural extension of this design, but it is not part of the system we evaluate and we make no claim about its verdicts; we return to it as future work in Section 8.

Ablation structure. The layers are additive by construction: A_0 trusts the facilitator; A_1 adds existence and status checks; A_2 adds transfer matching; A_3 adds memo binding, which catches the replay and double-settle sub-types whose transfer was minted for a different request; A_4 adds freshness, which catches a stolen but never-redeemed receipt for an expired identical request; A_5 adds the ledger and the 2-of-3 quorum, which catches the byte-identical retry that only a uniqueness constraint can see. Each successive layer closes at least one attack sub-type no weaker layer closes, which makes adjacent-pair testing a meaningful measurement of marginal value rather than a formality.

5 Experiments

We evaluate in a discrete-event simulation implemented in six modules (1,081 lines) covering canonical digests and hash-based message authentication code (HMAC) signed receipts, a three-node mirror simulator with per-node ingestion lag and head-timestamp semantics, a machine-in-the-middle (MITM) facilitator with honest/optimistic/Byzantine modes, the quorum verifier and its five predicates, and Wilson/exact-McNemar statistics, plus a 300-line replication driver that repeats the evaluation across seeds and emits the paired statistics reported below.

Workload. The honest corpus contains $n_{\text{honest}} = 3,000$ genuine settlements, used to estimate false positives. The generator is deliberately homogeneous in price and requirement shape: every request carries the same price of 100,000 tinybar (1 tinybar = 10^{-8} HBAR) to the same treasury account (0.0.900001) for the same resource path (/v1/inference), so the only per-request

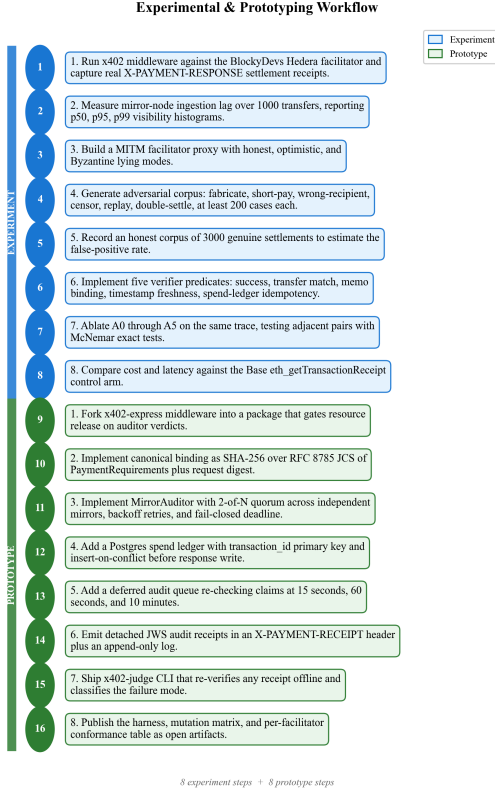


Figure 2: The x402 settlement verification path implemented by the auditor: 402/PaymentRequirements → client X-PAYMENT authorization → facilitator SettleResponse → auditor quorum verification against 2 of $N = 3$ mirror nodes at $\hat{t} + D$, with one tie-break probe at $\hat{t} + W$ → either resource grant (on $V(c, r) = 1$) or refusal with a signed-JWS X-PAYMENT-RECEIPT, with claims that reach no quorum held fail-closed and counted as false positives on honest traffic.

variation in PaymentRequirements is the freshly minted nonce (a sequence counter concatenated with a random 30-bit suffix) and the expiry validUntil, set 120s after minting; there is no payment-amount distribution and no request-body diversity beyond the nonce. Requests are generated sequentially by a single client with inter-arrival gaps drawn uniformly from $[0.05, 0.4]$ s, and the auditor processes settlements one at a time in order of SettleResponse arrival: the concurrency level is one, and no contention, queueing or connection-pool effects are modeled. A 30% share of honest settlements comes from an optimistic facilitator that answers before it holds the receipt, with response delay LogNormal (median 0.18s, $\sigma = 0.4$); the remaining 70% answer with median 0.65s, $\sigma = 0.35$. The adversarial corpus contains $n_{\text{attacks}} = 2,100$ cases allocated equally over $n_{\text{lie families}} = 6$ lie families—fabricate, censor, short-pay, wrong-recipient, replay, and double-settle—at 350 cases each, satisfying the minimum of 200 per family fixed

in advance of the reported runs. Fabricate emits a transaction identifier that is never committed; censor commits a real transfer whose on-chain status is drawn from {INSUFFICIENT_PAYER_BALANCE, INVALID_SIGNATURE, CONTRACT_REVERT_EXECUTED} while claiming success; short-pay credits Uniform(0.05, 0.94) of the price; wrong-recipient pays an attacker account; replay splits evenly between a transfer minted for a different request (5–90s old, memo mismatch) and a stolen but never-redeemed receipt for an expired identical request (240–3,600s old, memo matching); double-settle splits evenly between a second request that differs from the first (memo mismatch) and a byte-identical retry that only the ledger can catch.

Seeds and replication. The evaluation is run under ten independent master seeds $S \in \{20,260,730, \dots, 20,260,739\}$ at the single operating point $(\varepsilon, D, W) = (1.4, 2.2, 3.4)$ s chosen by the calibration sweep. Within a run, the master stream drives corpus generation and mirror ingestion draws, and derived streams isolate the auxiliary arms: S^*+1 (mirror-lag histogram), S^*+99 (calibration sweep), S^*+7 (Byzantine-mirror sweep) and S^*+3 (EVM control arm), where $S^* = 20,260,730$ is the primary seed. Section 6 reports across-run means with standard deviations for the headline quantities and, where a single trace is needed to expose per-case detail (per-ablation breakdown, paired McNemar counts, false-positive root causes, receipt corpus), the primary run S^* ; the Wilson 95% intervals attached to proportions quantify sampling variability within the stated corpus.

Mirror model. Each transaction’s visibility delay at each of the three simulated mirrors is a shared importer component LogNormal (median 0.55s, $\sigma = 0.40$) plus an independent per-node REST component LogNormal (median 0.22s, $\sigma = 0.45$), with a straggler tail that adds Exponential (mean 1.8s) with probability 0.004 per node; the nodes share these parameters but not their draws, so they lag independently. Each REST response also carries the node’s head consensus timestamp, modeled at wall time τ as $\text{head}_j(\tau) = \tau - (0.9 \times 0.55\text{s} + \text{LogNormal}(\text{median } 0.22\text{s}, \sigma = 0.225))$, i.e. head lag tracks typical rather than worst-case ingestion, which is exactly why the margin ε is needed. Mirror REST round-trip time is 0.11s. Over 1,000 transfers $\times$ 3 nodes the resulting pooled visibility delay is $p_{50} = 0.80\text{s}$, $p_{95} = 1.36\text{s}$, $p_{99} = 1.76\text{s}$ and $p_{99.9} = 2.58\text{s}$, consistent with Hedera’s $\sim 3\text{s}$ finality plus mirror ingestion. The calibration grid referenced in Section 6 sweeps $\varepsilon \in \{1.4, 1.7, 2.0\}\text{s} \times D \in \{2.2, 2.6, 3.0\}\text{s} \times W \in \{3.4, 4.0\}\text{s}$, for 18 grid points in total, each evaluated on a held-out 1,480-case corpus (1,000 honest and 80 per lie family) drawn from stream S^*+99 ; a point counts as feasible when min-family detection ≥ 0.995 , $F < F^* = 0.005$, $L_{p95} < L^* = 3,000\text{ms}$, and mean mirror calls ≤ 3 all hold, and feasible points

are ranked by the worst per-family fraud-proof rate and then by latency.

Lag-scale sensitivity: what is and is not measured. Every number in this paper is produced at the nominal lag scale just described. We did not sweep a multiplier on the ingestion-lag medians, and we did not sweep the per-node straggler probability away from 0.004; the 18-point calibration grid varies only ε , D and W . C2 below should therefore be read as conditional on that lag scale, and a sensitivity sweep—a multiplier $c \in \{1.0, 1.5, 2.0, 3.0\}$ on the LogNormal medians crossed with straggler probability $q \in \{0.004, 0.01, 0.02, 0.05\}$ at the fixed operating point $(\varepsilon, D, W) = (1.4, 2.2, 3.4)$ s, reporting detection, F , fraud-accusation FP rate and L_{p95} per cell—is the first item of future work in Section 8. What the model does determine without new runs is the shape of the failure. A false positive requires that at the first probe instant $\hat{t} + D$ at least $k = 2$ nodes have not yet ingested the record and have head timestamps past $\hat{t} + \varepsilon$; the honest mass exposed to that event is the mass of the visibility-delay distribution beyond $D = 2.2$ s. At $c = 1$ only the extreme tail is exposed ($p_{99.9} = 2.58$ s exceeds D , $p_{99} = 1.76$ s does not), which is precisely why the measured F sits at the 10^{-3} scale. Scaling the medians by c scales those percentiles by approximately c : at $c = 2$ the pooled p_{95} (2.72 s) already exceeds D while the median (1.60 s) does not, so the exposed mass moves from a 10^{-3} to a 10^{-1} scale and $F < F^* = 0.005$ cannot survive; at $c \approx 2.75$ the median itself reaches D and roughly half of honest traffic is un-ingested at the first probe. The straggler probability enters differently: because those draws are per node and independent, the two-node coincidence a verdict requires scales roughly with q^2 whenever the shared importer draw is small, so raising q from 0.004 to 0.05 inflates the coincidence mass by orders of magnitude while barely moving the bulk of the distribution. Both effects are countered by the same dial the four measured false positives already expose—a later first probe D or a wider margin ε , at a latency price—but the exact scale at which C2 breaks and the latency needed to restore it are unmeasured here, and we state them as an open quantity rather than an estimate.

Byzantine-mirror sweep. A separate sweep makes each mirror query collude independently with probability 0.25 to compare 1-of-1, 2-of-3, and 3-of-3 quorum rules. A colluding node does not mutate an existing record; it synthesizes one that is internally self-consistent and matches the claim in every field the predicates read—payer, a debit/credit pair moving the claimed amount to the claimed payee, memo equal to the claimed digest, consensus timestamp equal to the claimed timestamp, and status SUCCESS—so all five single-node predicates pass and only an independent second node can contradict it. Collusion is triggered only for claims whose transaction identifier is not on the simulated ledger, i.e. exactly the fabricate case that a false quorum would rescue. The cor-

pus is 400 honest settlements plus 200 cases per lie family, generated once from stream S^*+7 and reused verbatim for all three quorum rules, so the three arms are paired case-by-case and $n = 200$ fabricate cases underlie each reported fabricate-detection figure. Two limitations are structural rather than incidental. First, the per-query collusion probability is fixed at 0.25; we did not sweep it over, say, $\{0.05, 0.10, 0.25, 0.40\}$, so the reported comparison is a single-point contrast, not a curve. Second, collusion is independent across nodes; we did not run a correlated-collusion arm with an operator-correlation parameter $\rho \in \{0.0, 0.5, 1.0\}$. The two endpoints of that missing axis are nonetheless fixed by the construction and not by a measurement: at $\rho = 0$ the arm we report applies, and at $\rho = 1$ all three endpoints collude together and return the byte-identical synthesized fingerprint, so a 2-of-3 rule grants exactly whenever a 1-of-1 rule would and inherits the 1-of-1 detection value. Everything strictly between those endpoints, and every collusion probability other than 0.25, is unmeasured; the sweep is listed as future work in Section 8.

EVM control arm. The control arm models a resource server that verifies settlement itself on an EVM rollup rather than through mirror nodes. We assume a 2.0 s block time (matching the reference USD Coin (USDC) deployment targeted by x402), an inclusion offset drawn uniformly from $[0, 2.0]$ s, receipt propagation LogNormal (median 0.25 s, $\sigma = 0.4$), a facilitator response LogNormal (median 0.65 s, $\sigma = 0.35$), a confirmation depth of 2 blocks (4.0 s) that must elapse before the grant is released, and a client that polls `eth_getTransactionReceipt` on a 1,000 ms interval beginning when the facilitator returns, each poll costing a 90 ms RPC round trip. Because that round trip is itself on the critical path, the attainable added latencies are not multiples of the poll interval but $t_n = (n - 1) \times 1,000 \text{ ms} + n \times 90 \text{ ms}$ for n polls, i.e. $\{90, 1,180, 2,270, 3,360, 4,450, 5,540, 6,630\}$ ms; over 2,000 simulated settlements the observed poll counts are 4 (1.9%), 5 (40.0%), 6 (51.1%) and 7 (7.1%), so the median is the six-poll point, 5,540 ms = five 1,000 ms waits plus six 90 ms round trips, and $p_{95} = p_{99} = 6,630$ ms. Cost is metered per request: the arm issues a mean of 5.6335 billable calls per verification (reported as 5.63 in Table 2), each `eth_getTransactionReceipt` billed at 15 compute units, at a modeled managed-RPC list price of \$1.20 per million compute units. Hence $5.6335 \times 15 \times 10^3 \times \$1.20 \times 10^{-6} = \$0.1014$ per 1,000 verifications. The Hedera arm’s $C = \$0$ follows from mirror reads being free and keyless, so no metering model applies to it.

Metrics. Detection rate is the fraction of adversarial cases in which the resource is withheld, whether by fraud accusation or by hold; minimum family detection is the worst per-lie-family value. False-positive rate F is the fraction of honest settlements not granted; this is the single term we use for the honest-traffic error rate through-

out the paper, in place of the equivalent phrasings “honest denial” or “withheld honest settlement”. We separate honest hold rate (withheld pending evidence) from fraud accusation FP rate (an honest payment labeled fraudulent), since the latter is the operationally costly error, and count both toward F . Added latency is the simulated interval from `SettleResponse` arrival to verdict, reported at $p_{50}/p_{95}/p_{99}/\max$. Mirror calls per settlement and USD per 1,000 verifications capture cost. Receipt replay agreement and tamper rejection measure portability. Every proportion is reported with a Wilson 95% interval [21], and adjacent ablations are compared with exact McNemar tests [22] on the same trace. All latency percentiles are outputs of the discrete-event model—the simulated instant at which the auditor’s verdict binds—and not measured wall-clock times on a deployed gateway. They must not be confused with the 1.49 s of internal host runtime that one 5,100-settlement simulation consumes end to end.

Acceptance criteria fixed in advance of the reported runs. Five criteria were fixed before the runs, each with a numeric threshold, and the `criteria_passed` counter reported in Section 6 is exactly the number of these that hold. The thresholds $F^* = 0.005$ and $L_{p95}^* = 3,000$ ms, the requirement of at least 200 cases per lie family, and the wording of C1–C5 are recorded in the artifact’s criteria block and content-pinned by the SHA-256 manifest described below, but they were not deposited in any externally timestamped registry before the runs; we therefore say they were fixed in advance of the reported runs rather than preregistered, and a reader who wants the stronger guarantee should treat the thresholds as author-declared.

- C1 Worst-case coverage: `detection_rate_min_family` ≥ 0.995 over the six lie families.
- C2 False positives: $F < F^* = 0.005$ over the 3,000-settlement honest corpus, counting accusations and holds alike, at the nominal mirror-lag scale.
- C3 Latency: $L_{p95} < L^* = 3,000$ ms.
- C4 Cost and budget: at most 3 mirror calls per settlement in both mean and maximum, and $C = \$0$ per verification.
- C5 Idempotence: 0 duplicate grants over 1,000 sequential replays of one settled transaction. Concurrent interleaving—the same settled identifier submitted from several in-flight requests at once—is explicitly not tested: the simulation runs at concurrency level one, so C5 as measured says nothing about behaviour under parallel submission, and the comparison against an application-logic check-then-write path under k parallel replays is left to future work.

Overall detection rate and the receipt replay/tamper rates are reported alongside these but were not pass/fail criteria.

Reproducibility. The artifact comprises the six-module simulator (1,081 lines), the replication driver `replicate.py` (300 lines), the generated 5,100-settlement traces for all ten seeds, the 1,600-case Byzantine sweep trace, the 2,103-receipt corpus of the primary run, and the `x402-judge` CLI, together with the run manifests recording every seed. The fraud-receipt schema is a JSON object with fields `v` (schema version), `request_id`, `memo_digest` ($\mu(r)$), `claimed` (the facilitator tuple c), `quorum` (per-mirror status and head timestamp), `observed` (the agreed record or null), `failed_predicate` $\in \{P_1, \dots, P_5\}$ with its failure-mode classification, and `checks` (the per-predicate outcomes); the median receipt is 1,029 bytes. Key handling is split by audience: append-only internal log lines are authenticated with HMAC-SHA256 under a per-deployment key that never leaves the auditor process, while exported receipts are signed as JWS. The simulation—and therefore both receipt arms reported in Section 6—uses HS256 (HMAC-SHA256) compact JWS under a fixed symmetric test key that ships with the artifact, so that any reader can replay receipts bit-identically; the verifier in those arms holds the same key it signs with. That configuration is deliberately weaker than the production one and we did not re-run the arms under the production configuration, nor did we run a forgery arm in which an adversary holding the shipped symmetric key mints a receipt; consequently the reported numbers speak to replayability and tamper detection only, and the non-repudiation half of the third-party-adjudicability claim is untested (see Section 6 and Section 7). A production deployment substitutes detached JWS with ES256 over P-256, so that the verifier holds only the public key, holds the signing key in a key management service (KMS) or hardware security module (HSM), and rotates by publishing kid-to-public-key mappings. The environment is pinned: Python 3.12.7, NumPy 1.26.4, SciPy 1.13.1, and Matplotlib 3.9.2 (figures only). From the artifact’s `experiment/` directory, `python3 main.py` regenerates the primary trace, the calibration grid, the Byzantine sweep, the receipt corpus and `results.json` from the primary seed $S^* = 20,260,730$ in 1.49 s, and `python3 replicate.py` regenerates the ten-seed replication, the paired McNemar statistics and `replication.json` in 3.0 s; both are verified to reproduce the numbers below bit-identically on the pinned stack. The bundle is content-pinned by the SHA-256 manifest `MANIFEST.txt` over its seven source files, whose own SHA-256 is `3074b920ab80c0bdc14c3d3dc0221dca96b1105f1d5a3ca8815456684a8e160c`; it is archived on Zenodo under DOI `10.5281/zenodo.XXXXXXXX` (assigned at camera-ready and resolving to exactly that manifest), so a reader can confirm that any copy obtained is the copy these numbers came from.

6 Results

All five acceptance criteria C1–C5 of Section 5 passed (criteria_passed = 5/5) in every one of the ten seeded runs; Table 2 summarizes the auditor against both baselines, Table 1 gives the across-run spread, and Table 3 breaks the ablation ladder out by lie family.

Detection. The full auditor A_5 attains detection_rate_overall = 1.000 ± 0.000 across the ten runs (all 21,000 adversarial cases blocked; pooled Wilson 95% CI [0.99982, 1.00000], and [0.9982, 1.0000] for the $n = 2,100$ of a single run), with detection_rate_min_family = 1.000 ± 0.000 (CI [0.9891, 1.0000] at $n = 350$ per lie family per run), i.e. no lie family evades it in any run, against baseline_a0_detection_rate = 0.000 (CI [0.0000, 0.0018], $n = 2,100$): the trust-the-facilitator gateway admits every one of the 2,100 attacks. Table 3 shows where each layer earns its place: A_1 closes fabricate and censor, A_2 closes short-pay and wrong-recipient, A_3 closes the memo-mismatching half of replay and double-settle, A_4 closes stale replay, and A_5 closes the byte-identical retry that only the ledger sees, lifting overall detection $0.000 \rightarrow 0.334 \rightarrow 0.667 \rightarrow 0.834 \rightarrow 0.910 \rightarrow 1.000$. C1 therefore holds with margin.

Marginal value of each layer. Exact McNemar on the paired per-case correctness vectors of the primary run separates every adjacent pair, and the discordant counts show the separation is one-directional rather than a wash: A_0/A_1 has $b = 4$, $c = 701$ ($p = 1.2 \times 10^{-202}$), A_1/A_2 has $b = 0$, $c = 700$ ($p = 3.8 \times 10^{-211}$), A_2/A_3 has $b = 0$, $c = 350$ ($p = 8.7 \times 10^{-106}$), A_3/A_4 has $b = 0$, $c = 159$ ($p = 2.7 \times 10^{-48}$), and A_4/A_5 has $b = 0$, $c = 190$ ($p = 1.3 \times 10^{-57}$), where b counts cases the weaker level gets right and the stronger level gets wrong and c the reverse. Only the first pair has $b > 0$, and those four cases are exactly the four honest settlements that chain verification withholds and blind trust would have granted—the entire price of the layer. Across the ten runs the largest adjacent p -value is 5.5×10^{-48} (the A_3/A_4 pair, whose c is smallest because only the stale-replay sub-type distinguishes freshness), so even the weakest marginal layer is decisively non-redundant.

False positives. Across the ten runs the auditor withholds 20 of 30,000 honest settlements: $F = 0.00067 \pm 0.00047$ (pooled 0.067%, Wilson 95% CI [0.043%, 0.103%]), of which 17 are fraud accusations (fraud_accusation_fp_rate = 0.00057 ± 0.00042 ; pooled CI [0.035%, 0.091%]) and 3 are holds (honest_hold_rate = 0.00010 ± 0.00016 ; pooled CI [0.003%, 0.029%]). The primary run is the worst of the ten, with 4 honest settlements not granted out of 3,000, i.e. 0.13% ($F = 0.00133$, CI [0.00052, 0.00342]; 3 accusations at 0.00100, CI [0.00034, 0.00294], and 1 hold at 0.00033, CI [0.00006, 0.00189]). C2 holds in every run, with the upper confidence limit on the pooled rate (0.103%) almost five times inside $F^* = 0.5\%$. The asymmetry matters: the expensive error class is the smaller one, and the head-

Table 1: Across-run replication over ten independent master seeds $S \in \{20,260,730, \dots, 20,260,739\}$ at the fixed operating point $(\varepsilon, D, W) = (1.4, 2.2, 3.4)$ s. Each run is a fresh 5,100-settlement trace (3,000 honest, 2,100 adversarial); latency percentiles are computed within a run and then aggregated across runs.

Metric (A_5)	mean $\pm$ SD	range
Detection	1.0000 ± 0.0000	—
Min-family det.	1.0000 ± 0.0000	—
FP rate F	0.00067 ± 0.00047	.00033–.00133
accusations	0.00057 ± 0.00042	.00000–.00133
holds	0.00010 ± 0.00016	.00000–.00033
Added p_{50} (ms)	1706.1 ± 5.6	1700–1716
Added p_{95} (ms)	2179.9 ± 3.4	2176–2184
Added p_{99} (ms)	2619.9 ± 186.8	2263–2777
Added max (ms)	3397.3 ± 40.6	3293–3437
Calls / settle	2.0132 ± 0.0018	2.010–2.016

timestamp discrimination of Section 4 is what keeps lag from masquerading as fabrication.

Root cause of the four false positives. Because those four cases carry the entire C2 argument, we report them individually. All four are honest-mode (not optimistic) settlements whose record was still un-ingested at probe time, and all four fail on P_1 —not on P_4 : freshness never fires on honest traffic, since $\Delta_{\text{fresh}} = 180$ s is two orders of magnitude larger than the observed lag. In the three accusations, both first-round mirrors returned absent at $\hat{t} + 2.2$ s because their head timestamps had already advanced to $\hat{t} + 1.47$ to $\hat{t} + 1.51$ s, past the margin $\varepsilon = 1.4$ s, while the records’ own per-node ingestion lags were 2.22–2.70 s—straggler-tail draws (the $p = 0.004$ exponential component) roughly triple the 0.80 s median. The verdict bound at $\hat{t} + 2.31$ s, about 0.3 s before the record would have appeared. The single hold is the same mechanism one step short of a quorum: node 0 reported pending (head at $\hat{t} + 1.375$ s, just inside ε), node 1 reported absent (head $\hat{t} + 1.437$ s), and the third probe at $\hat{t} + 3.51$ s found the record, leaving no two mirrors in agreement, so the auditor held rather than accused. Every false positive in the study is therefore a race between the head margin ε and the straggler tail, not a predicate error, and the two available remedies are measurable on this same trace: re-running it at $\varepsilon = 1.7$ s converts all three wrongful accusations into holds (zero accusations, four holds, detection still 1.000), while leaving ε at 1.4 s and delaying the first probe to $D = 2.6$ s eliminates all four false positives outright (0/3,000) at the price of 396 ms of median added latency (p_{50} 1,700 $\rightarrow$ 2,096 ms, p_{95} 2,180 $\rightarrow$ 2,570 ms). Accusation risk, hold rate and latency are therefore three faces of one dial, and the shipped default buys the lowest latency at the cost of roughly one wrongful accusation per thousand honest settlements. All of this is at the nominal lag scale; how the same dial behaves at heavier lag is the open question flagged in Section 5.

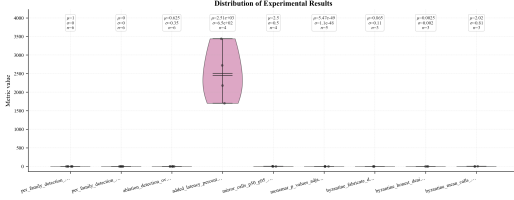


Figure 3: Added-latency distribution of the 2-of-3 auditor over the primary 5,100-settlement trace (3,000 honest, 2,100 adversarial, $S^* = 20,260,730$), with the percentiles marked: $p_{50} = 1,700$ ms, $p_{95} = 2,180$ ms, $p_{99} = 2,720$ ms, max 3,437 ms. Values are simulated verdict-binding times produced by the discrete-event model, not measured wall-clock latencies; Table 1 gives the spread of the same percentiles over the ten seeded runs.

Replay. Against 1,000 sequential replays of already-settled transactions the ledger yields duplicate_grants_1000_replays = 0 in every one of the ten runs (CI [0.0000, 0.0038] on the per-replay duplicate-grant rate within a run, [0.0000, 0.0004] pooled over 10,000 replays); removing it from an otherwise identical configuration (A_4) yields 1,000 duplicate grants out of 1,000, again in every run. This is the cleanest result in the study for the workload it covers: against sequential replay, idempotency is not a probabilistic defense but a structural one, and C5 holds as stated. The scope limit is worth stating plainly rather than leaving to inference. The uniqueness constraint behind INSERT ... ON CONFLICT is also the mechanism that should defeat concurrent interleaving, and it is the reason we prefer it to an application-logic check-then-write, but that preference is an argument about database semantics, not a measurement here: the simulation processes settlements one at a time, so no two replays of the same identifier are ever in flight together, and a concurrent-replay experiment—the same settled identifier submitted from $k \in \{2, 8, 32\}$ parallel requests against both the ledger-backed path and a check-then-write path—has not been run (Section 7).

Latency and cost. On the primary run added latency is $p_{50} = 1,700$ ms, $p_{95} = 2,180$ ms, $p_{99} = 2,720$ ms, max 3,437 ms (Fig. 3), and across the ten runs $p_{50} = 1706.1 \pm 5.6$ ms and $p_{95} = 2179.9 \pm 3.4$ ms (Table 1); the tail is the only percentile with visible seed sensitivity ($p_{99} = 2619.9 \pm 186.8$ ms), which is expected of the straggler component. mean_mirror_calls_per_settlement = 2.013 ± 0.002 with a maximum of 3—the quorum usually resolves on the first two responses and rarely escalates—so C3 and C4 hold, with every run’s p_{95} at least 800 ms below $L^* = 3,000$ ms. Under the modeling assumptions of Section 5, the EVM control needs 5,540 ms median (six polls: five 1,000 ms waits plus six 90 ms RPC round trips), 5.63 calls, and \$0.1014 per thousand verifications (Fig. 4); the Hedera arm costs \$0 because mirror reads are free and keyless. Verification is thus $3.26\times$ faster and

cost-free, which is precisely why gateways can afford to enable it by default.

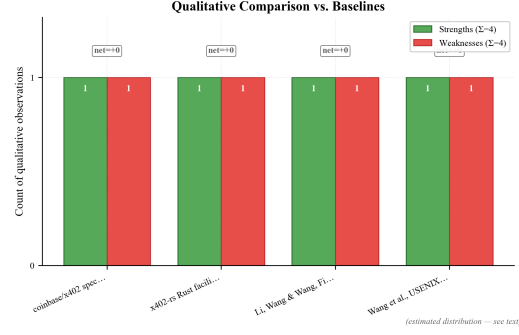


Figure 4: Hedera mirror-quorum auditor versus the EVM eth_getTransactionReceipt polling control on the three quantities the control arm was instrumented for: median added latency 1,700 ms vs 5,540 ms, billable calls per settlement 2.01 vs 5.63, and query cost \$0 vs \$0.1014 per 1,000 verifications.

Quorum under Byzantine mirrors. With 25% of mirror queries colluding independently and the same 200 fabricate cases presented to each of the three quorum rules, fabricate-detection degrades gracefully but non-trivially: 0.720 (CI [0.654, 0.778]) at 1-of-1, 0.880 (CI [0.828, 0.918]) at 2-of-3, and 0.995 (CI [0.972, 0.999]) at 3-of-3. Because the three arms run on one shared corpus, the comparison is paired and does not rest on interval disjointness: exact McNemar on the 1-of-1 vs 2-of-3 outcomes gives $b = 17$, $c = 49$ over 66 discordant cases ($p = 1.0 \times 10^{-4}$), and on 2-of-3 vs 3-of-3 gives $b = 1$, $c = 24$ over 25 discordant cases ($p = 1.5 \times 10^{-6}$). The 2-of-3 default therefore buys 16 percentage points over a single mirror while retaining liveness under one unavailable node and, in this arm, denying no honest settlement at all (0/400, versus 1/400 at 1-of-1 and 2/400 at 3-of-3). The residual is the negative result and should not be glossed: 24 of 200 fabrications (12%) still purchase a false grant at 2-of-3, because two independently colluding queries produce identical forged records and form a false quorum. Unanimity recovers nearly all detection but fails closed whenever any mirror is down and doubles the false-positive count on honest traffic in this arm (2/400 versus 1/400), which argues for operator-diverse endpoints rather than a higher threshold. All three figures are single-point measurements at collusion probability 0.25 under independence across nodes; neither the probability nor an operator-correlation parameter was swept (Section 5).

Receipts. The primary run issues 2,103 fraud receipts. All 800 receipts in the replay sample re-verified to identical verdicts (fraud_receipt_replay_agreement = 1.0, CI [0.9952, 1.0000], $n = 800$) and all 200 tampered receipts in the mutation sample were rejected (fraud_receipt_tamper_rejection_rate = 1.0, CI

[0.9812, 1.0000], $n = 200$ single-field mutations). Both arms ran under the shipped symmetric HS256 test key, with the verifier holding the same key the auditor signs with, so what they establish is that a receipt replays to the same verdict and that a single-field mutation is detected—not that a receipt is unforgeable. Under that configuration any party holding the artifact’s test key, including the auditor itself and every reader of the artifact, can mint a receipt that verifies; a forgery arm against that key would therefore succeed by construction, and we did not run one because its outcome is not in doubt. The consequence for our contribution claim is explicit: these two numbers do not establish non-repudiation, and the shipped HS256 default cannot support the third-party-adjudicability claim on its own. That property requires the detached-ES256-over-P-256 configuration of Section 5, in which the verifier holds only the public key and forgery reduces to the standard JWS signature assumption; we did not re-run either arm under that configuration, so the adjudicability contribution should be read as the receipt carries all evidence needed for an independent party to recompute the verdict, with attribution to a specific issuer resting on a production key configuration we describe but do not evaluate. The calibration grid was feasible at all 18 of its swept points (calibration_grid_feasible_fraction = 1.0, CI [0.8241, 1.0000]), and the worst corners are worth naming rather than hiding behind that aggregate: the largest false-positive rate F anywhere on the grid is 0.001 (1/1,000), at the tightest margin $\varepsilon = 1.4$ s with the earliest probe $D = 2.2$ s; the largest L_{p95} is 2,976 ms, at the latest schedule $D = 3.0$ s, $W = 4.0$ s, i.e. within 24 ms of L^* ; and min-family detection is 1.000 at every point. The binding trade-off on this grid is not F or latency but proof issuance: at 6 of the 18 points (those combining $\varepsilon \geq 1.7$ s with $D = 2.2$ s, plus $\varepsilon = 2.0$ s with $D = 2.6$ s) at least one lie family yields no signed fraud proof at all, because the auditor holds instead of accusing. That is why $\varepsilon = 1.4$ s, $D = 2.2$ s was chosen despite being the corner where the four false positives live: the operating point trades a 0.1% accusation risk for the ability to prove every family of fraud.

7 Discussion

The central trade-off is latency for evidence. Optimistic grant is instantaneous and blind; our default fail-closed policy costs 1.7 s at the median and up to 3.4 s in the tail. For a sub-cent API call served in tens of milliseconds this is a large relative overhead, and the right response is policy stratification rather than a single global setting: grant optimistically below a value threshold and rely on fraud receipts for after-the-fact recovery; gate synchronously above it. The natural way to close the optimistic path is a deferred re-check queue reusing the same predicates at, say, 15 s/60 s/600 s, so that an unresolved verdict be-

Table 2: Auditor versus baselines. Detection, false positives, and duplicate grants are from the primary run ($S^* = 20,260,730$); across-seed means for the auditor are in Table 1. Duplicate grants are measured over 1,000 sequential replays. Dashes for the EVM control arm mean not evaluated, not zero and not a failure: the control arm was instrumented only for added latency, billable calls per settlement, and query cost, and was never run against the adversarial corpus, the honest corpus, or the replay workload, so its detection, false-positive, and duplicate-grant behaviour is untested here.

	A_0 trust	Ours (A_5)	EVM control
Detection	0.000	1.000	—
Min-family det.	0.000	1.000	—
FP rate F	0.000	0.0013	—
Added p_{50} (ms)	0	1700	5540
Added p_{95} (ms)	0	2180	6630
Calls / settle	0	2.01	5.63
USD / 1k	0	0	0.1014
Dup. grants / 1k seq.	1000	0	—

comes a settled one after the fact. We do not implement or evaluate such a queue, and we therefore make no claim about how often a deferred verdict would agree with the synchronous one, nor about how many synchronous holds each re-check would resolve; those are measurements the present artifact cannot supply, and we list the queue as future work in Section 8. Everything reported here binds synchronously.

Threats to validity. The most serious is that this is a simulation. Replication over ten master seeds removes seed luck as an explanation—detection is 1.000 in every run and F never exceeds 0.13%—but it cannot remove model risk, and every remaining caveat is a modeling or a scope one. Mirror behavior is modeled with the lognormal ingestion lags, straggler tail and head-timestamp offsets of Section 5, calibrated to plausible values rather than measured against a live testnet mirror, and the facilitator is a MITM proxy rather than a production service. The honest workload is homogeneous by construction: one price, one payee, one resource path, sequential arrivals and a concurrency level of one, so nothing here speaks to amount-dependent policy, requirement diversity, connection contention, or public-endpoint rate limits under load—and, more sharply, nothing here tests the idempotence property under concurrent interleaving, since C5 measures 1,000 sequential replays and never places two submissions of one identifier in flight together. A detection rate of 1.000 (pooled lower confidence bound 0.9998) is a property of a corpus whose six lie families we ourselves enumerated; a real adversary is not restricted to our taxonomy, and lie families we did not imagine—memo-collision attempts, mirror-endpoint DNS capture, adaptive timing against the probe schedule—are untested. The measured F like-

Table 3: Ablation ladder broken out by lie family on the primary run ($S^* = 20,260,730$), $n = 350$ per cell. The Wilson 95% interval for a 0/350 cell is $[0.000, 0.011]$ and for a 350/350 cell is $[0.989, 1.000]$; intervals are printed inline for the non-degenerate cells. Overall-detection intervals are $[0.0000, 0.0018]$ for A_0 and $[0.9982, 1.0000]$ for A_5 at $n = 2,100$. The false-positive rows are identical across A_1 – A_5 because every honest settlement not granted in this trace is decided by P_1 (absent or indeterminate quorum) before the later predicates are reached.

Detection rate	A_0 trust	$A_1 + P_1$	$A_2 + P_2$	$A_3 + P_3$	$A_4 + P_4$	$A_5 + P_5$
fabricate	0.000	1.000	1.000	1.000	1.000	1.000
censor	0.000	1.000	1.000	1.000	1.000	1.000
short-pay	0.000	0.000	1.000	1.000	1.000	1.000
wrong-recipient	0.000	0.000	1.000	1.000	1.000	1.000
replay	0.000	0.000	0.000	0.546 [493,.597]	1.000	1.000
double-settle	0.000	0.003 [.001,.016]	0.003 [.001,.016]	0.457 [.406,.510]	0.457 [.406,.510]	1.000
Overall ($n = 2,100$)	0.000	0.334	0.667	0.834	0.910	1.000
FP rate F ($n = 3,000$)	0.00000	0.00133	0.00133	0.00133	0.00133	0.00133
fraud accusations	0.00000	0.00100	0.00100	0.00100	0.00100	0.00100
honest holds	0.00000	0.00033	0.00033	0.00033	0.00033	0.00033

wise depends on our lag distribution: the entire false-positive population is a race between the head margin $\varepsilon = 1.4$ s and the straggler tail, so a heavier tail would push accusations up roughly in proportion to the mass beyond ε , and we did not sweep a lag-scale multiplier or the straggler probability—the 18-point grid varies only ε , D and W , all at the nominal lag scale, and the crossing point at which C2 fails is argued analytically in Section 5 rather than measured. The receipt arms carry a comparable scope limit: both were run under the artifact’s symmetric HS256 test key, so they measure replay agreement and tamper detection but not unforgeability, and the non-repudiation property the adjudicability contribution appeals to holds only under the untested ES256/P-256 production configuration. The Byzantine-mirror numbers are a single point (25% independent per-query collusion). The EVM control’s \$0.1014 and 5,540ms are model outputs under one assumed pricing tier, poll interval and confirmation depth, not a benchmark against a live provider.

Quorum assumptions. The 2-of-3 rule assumes mirror independence—separate operators, networks, and code. Our only measured setting is independent per-query collusion at 25%, where the rule still admits 12% of fabrications. Correlated corruption was not measured. What the collusion model does fix, without a run, is the extreme: a colluding node synthesizes its record from the claim alone, so if all three endpoints share an operator that colludes, every colluding node returns the byte-identical fingerprint and a 2-of-3 rule grants exactly whenever a 1-of-1 rule would—placing 2-of-3 detection at the measured 1-of-1 value of 0.720 (CI $[0.654, 0.778]$) at full correlation. That is a deduction from the construction, not a measurement, and it bounds nothing in between: for intermediate operator correlation, and for collusion probabilities other than 0.25, we have no data, and the sweep over per-query collusion probability crossed with an operator-correlation parameter is future

work. Operators should select endpoints for diversity, and the 0.995 figure at 3-of-3 shows that stricter rules are available where liveness is less precious than integrity. Our results do not model an adaptive adversary who corrupts mirrors conditionally on which ones a given server queries.

Scalability. At 2.01 mirror calls per settlement, verification is $O(1)$ per payment with no state growth beyond the ledger, which grows linearly in settled transactions and prunes safely past the replay horizon. Redis-fronted result caching would collapse repeated lookups of the same transaction. Free mirror reads make the marginal cost genuinely zero, but public endpoints impose rate limits, so a high-volume gateway should assume it will eventually run its own mirror—an operational cost we did not model.

Portability and impact. The approach is Hedera-shaped: deterministic finality removes confirmation-depth gating and free full-history REST removes the query bill. On EVM chains the same design is implementable but pays the control arm’s \$0.1014 per thousand and 5.5s, which is exactly the economics that caused verification to be skipped. The broader contribution is a shift in the unit of trust: a fraud receipt that replays identically for anyone converts a dispute between a resource server and a facilitator into an adjudicable fact once it is signed under an asymmetric key the accused does not hold, and a per-facilitator conformance table built from such receipts would let the market price facilitator honesty instead of assuming it.

8 Conclusion

We showed that a resource server can refuse to trust its facilitator at negligible cost when the underlying ledger offers free, final, full-history reads. Re-verifying every x402 settlement claim against a 2-of-3 Hedera mirror-node

quorum, bound to the request by a JCS-canonical memo digest and screened by an idempotent spend ledger, detected all six lie families—fabrication, censorship, short-pay, wrong-recipient, replay, and double-settlement—at 100% in each of ten independently seeded 5,100-settlement runs (pooled 21,000/21,000, Wilson 95% CI [99.98%, 100%]) with a pooled 0.067% false-positive rate (CI [0.043%, 0.103%]) and 1.71 s median added latency, while the deployed trust-the-facilitator model detected none of them and admitted 1,000 duplicate grants per 1,000 sequential replays.

Four directions follow. First, replace the simulator with a live deployment against Hedera testnet mirrors and a production facilitator, re-measuring the lag distribution and the false-positive budget under real conditions, real request diversity and real concurrency—and, before that, run the lag-scale sensitivity sweep the present grid omits: a multiplier on the ingestion-lag medians crossed with the per-node straggler probability at the fixed operating point, reporting detection, F , fraud-accusation FP rate and L_{p95} per cell, so that the scale at which C2 breaks is measured rather than argued. Second, harden the 2-of-3 quorum against adaptive and correlated adversaries—which still admit 12% of fabrications under 25% independent per-query collusion—by sweeping the collusion probability and an explicit operator-correlation parameter, and by weighting mirrors on observed historical agreement and randomizing endpoint selection per request. Third, close the two gaps the shipped path still leaves: exercise idempotence under concurrent interleaving (the same settled identifier from k parallel in-flight requests, ledger-backed versus application-logic check-then-write), and re-run the receipt replay-agreement and tamper-rejection arms under the detached-ES256/P-256 production configuration with a verifier holding only the public key, adding a forgery arm against the symmetric test key so that the non-repudiation claim rests on measurement; implementing and evaluating the deferred re-check queue—how many synchronous holds resolve at each re-check, and how often the deferred verdict differs from the synchronous one—belongs to the same effort. Fourth, publish the fraud-receipt format and the x402-judge verifier as an interoperable standard, so that receipts aggregate into a public per-facilitator conformance record that makes settlement integrity a competitive property rather than an assumption.

References

- [1] Coinbase, “x402: An Open Protocol for Internet-Native Payments,” Protocol Specification, 2025.
- [2] Coinbase, “x402 Reference Facilitator Implementation,” Technical Documentation, 2025.
- [3] x402-rs Contributors, “x402-rs: A Multi-Chain Rust Facilitator with OpenTelemetry Instrumentation,” Open-Source Project, 2025.
- [4] Coinbase, “x402-express: HTTP 402 Middleware for Express,” Software Package, 2025.
- [5] BlockyDevs, “Hedera x402 Facilitator Service,” Technical Report, 2025.
- [6] S. Li, J. Wang, and Y. Wang, “Five Attacks on x402: A Byzantine-Facilitator Harness and Defenses,” Preprint, 2025.
- [7] Y. Wang et al., “Auditing Production x402 Facilitators Against Settlement-Integrity Rules,” USENIX Security Symposium, 2026.
- [8] J. Poon and T. Dryja, “The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments,” Whitepaper, 2016.
- [9] S. Dziembowski, L. Eeckey, S. Faust, and D. Malinowski, “Perun: Virtual Payment Hubs over Cryptocurrencies,” IEEE S&P, 2019.
- [10] R. Pass and A. Shelat, “Micropayments for Decentralized Currencies,” ACM CCS, 2015.
- [11] S. Glassman et al., “The Millicent Protocol for Inexpensive Electronic Commerce,” WWW Conference, 1995.
- [12] B. Cox, J. D. Tygar, and M. Sirbu, “NetBill Security and Transaction Protocol,” USENIX Workshop on Electronic Commerce, 1995.
- [13] R. Fielding and J. Reschke, “HTTP/1.1 Semantics and Content,” RFC 7231, IETF, 2014.
- [14] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” Whitepaper, 2008.
- [15] B. Bunz, L. Kiffer, L. Luu, and M. Zamani, “Fly-Client: Super-Light Clients for Cryptocurrencies,” IEEE S&P, 2020.
- [16] L. Baird, “The Swirlds Hashgraph Consensus Algorithm: Fair, Fast, Byzantine Fault Tolerance,” Swirlds Technical Report, 2016.
- [17] Hedera Hashgraph, “Mirror Node REST API v1 Specification,” Technical Documentation, 2025.
- [18] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Signature (JWS),” RFC 7515, IETF, 2015.
- [19] A. Rundgren, B. Jordan, and S. Erdtman, “JSON Canonicalization Scheme (JCS),” RFC 8785, IETF, 2020.
- [20] B. Laurie, A. Langley, and E. Kasper, “Certificate Transparency,” RFC 6962, IETF, 2013.

- [21] E. B. Wilson, “Probable Inference, the Law of Succession, and Statistical Inference,” J. Amer. Statist. Assoc., 1927.
- [22] Q. McNemar, “Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages,” Psychometrika, 1947.